Les différents types de navigateurs

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Le navigateur par onglets (TabNavigator) **4**

III - Le navigateur en panneau (DrawerNavigator) **8**

IV - Navigation avancée **11**

V - Synthèse **16**

VI - Auto évaluation **19**

A. Exercice : Concevoir la navigation d'une application mobile de gestion d'événements 19

B. Quiz 20

Solutions des exercices **24**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 h

Environnement de travail : Une application React Native initialisée ou utiliser https://snack.expo.io/

Pré-requis : Avoir vu le module sur les bases de la navigation dans une application React Native

Contexte

Pour apprendre le fonctionnement de base de la navigation dans une application React Native, nous avons étudié les navigateurs les plus proches de la gestion de la navigation dans une page web : les navigateurs en pile (`StackNavigator`).

Cependant, ce ne sont pas les seuls navigateurs disponibles dans une application mobile et, bien qu'on puisse inventer son propre type de navigateur, laissant libre cours à l'imagination des designers et des développeurs, `react-navigation` nous propose, en plus du navigateur en pile, deux autres types de navigateur : les navigateurs en tables (`TabNavigator`) et les navigateurs panneaux (`DrawerNavigator`).

Une application mobile est généralement la combinaison d'au moins deux de ces trois navigateurs et permet de définir des parcours utilisateurs complets et professionnels.

Tous droits réservés à STUDI - Reproduction interdite

II Le navigateur par onglets (TabNavigator)

Objectifs

- Voir les différents navigateurs par onglets
- Comprendre le contexte d'utilisation

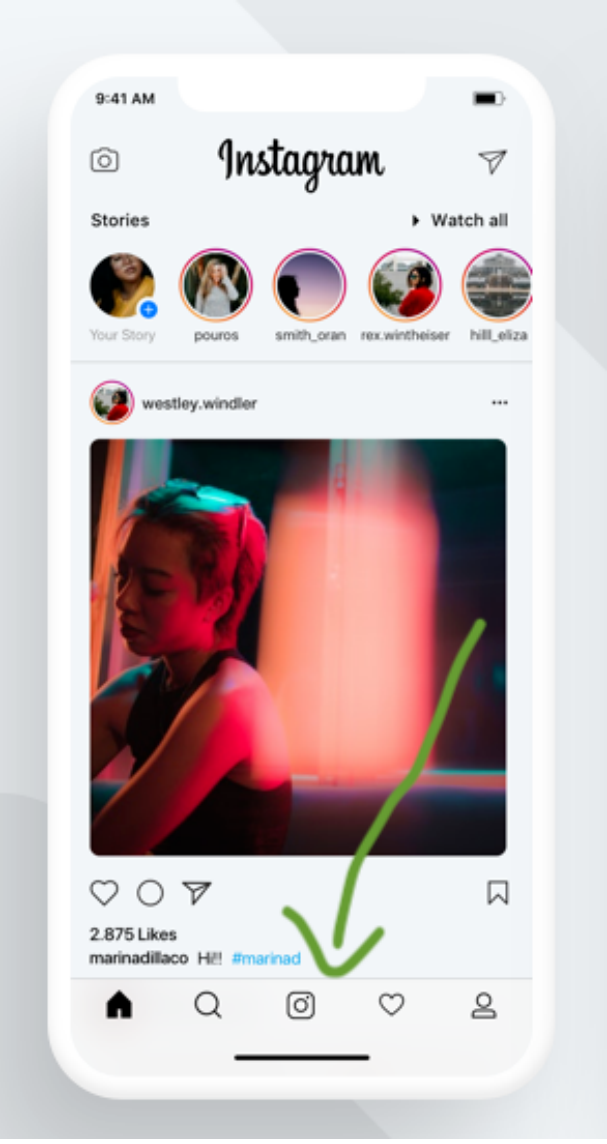
Mise en situation

Le navigateur par onglets (`TabNavigator`) est un navigateur très populaire pour la navigation mobile. En effet, les interfaces étant très petites à cause du format des écrans de téléphone et adaptées pour une utilisation tactile, cela en fait un pilier de choix dans la réalisation d'interfaces pratiques d'utilisation. Nous verrons également que le navigateur par onglets existe en plusieurs déclinaisons et que l'on peut personnaliser certains de ses éléments.

Qu'est-ce qu'un navigateur par onglets et quand l'utiliser ?

Le navigateur par onglets permet d'afficher un menu sur une ligne avec X éléments, on peut ensuite *swiper* ou cliquer sur un élément de ce menu pour afficher un écran correspondant.

Par exemple, l'application mobile d'Instagram utilise un navigateur par onglets en bas de son écran avec 5 éléments de navigation : ce sont des liens vers nos écrans.



Tous droits réservés à STUDI - Reproduction interdite

Pour utiliser un navigateur par onglets, il faut installer le module lié à ce dernier, `@react-navigation/bottom-tabs`. Mais, d'abord, il faut s'assurer d'avoir `@react-navigation/native` qui a normalement été installé dans le module précédent. Adaptons donc la commande d'installation suivante en fonction de nos besoins : `npm install @react-navigation/native @react-navigation/bottom-tabs`.

Voici un exemple très simple d'utilisation d'un navigateur par onglets qui s'attache en bas d'écran : il affiche deux écrans dans l'interface, un pour la page d'accueil et l'autre pour le profil de l'utilisateur.

Exemple

```
import * as React from 'react';
import { Text, View, StyleSheet } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

// Notre premier écran
function HomeScreen() {
  return (
    <View style={styles.containers}>
      <Text>Home</Text>
    </View>
  );
}

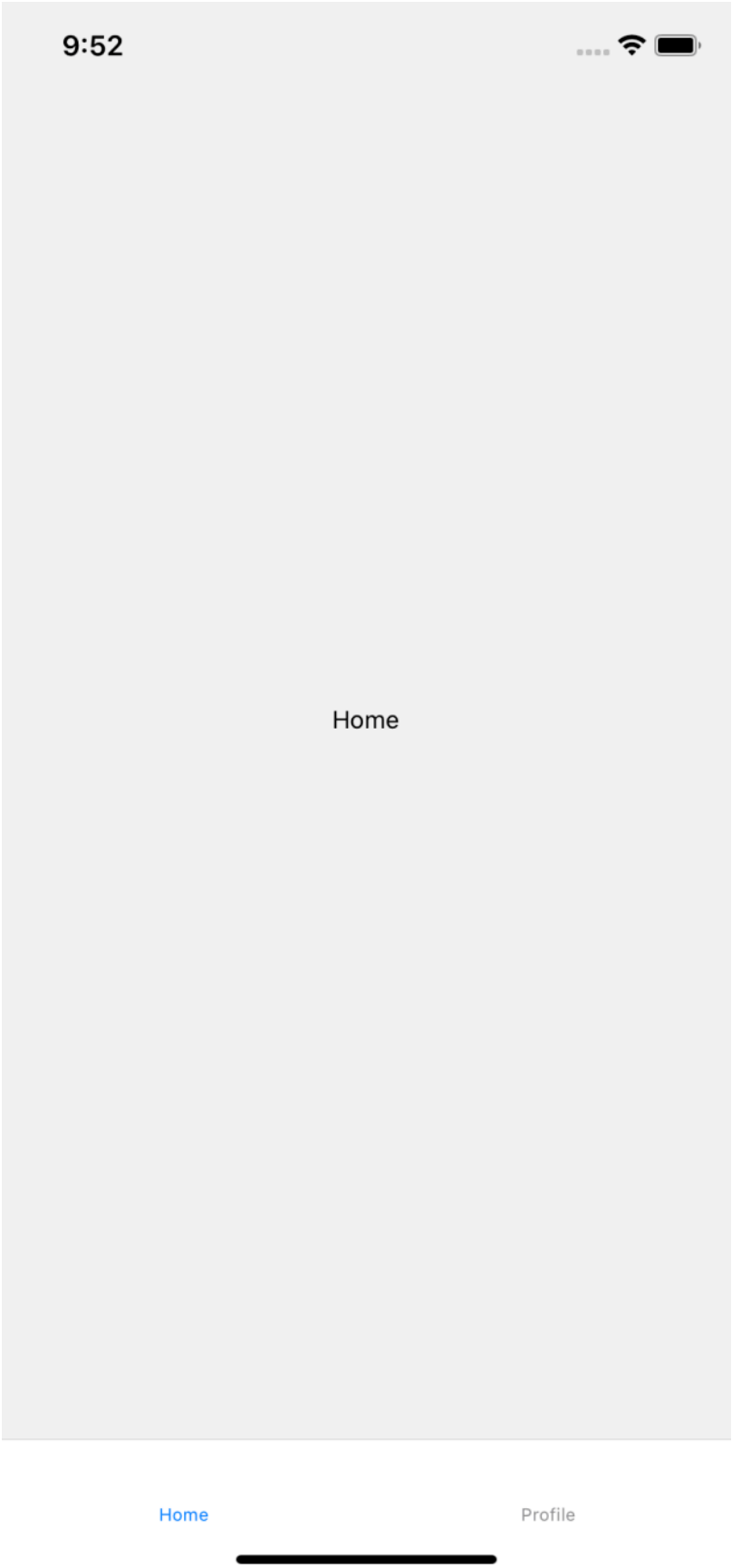
// Notre deuxième écran
function ProfileScreen() {
  return (
    <View style={styles.containers}>
      <Text>Profile</Text>
    </View>
  );
}

// Des styles communs pour l'affichage
const styles = StyleSheet.create({
  containers: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
});

// Notre navigateur en tables
const Tab = createBottomTabNavigator();

// Et notre application ainsi que la définition des écrans
export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen name="Home" component={HomeScreen} />
        <Tab.Screen name="Profile" component={ProfileScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

Tous droits réservés à STUDI - Reproduction interdite

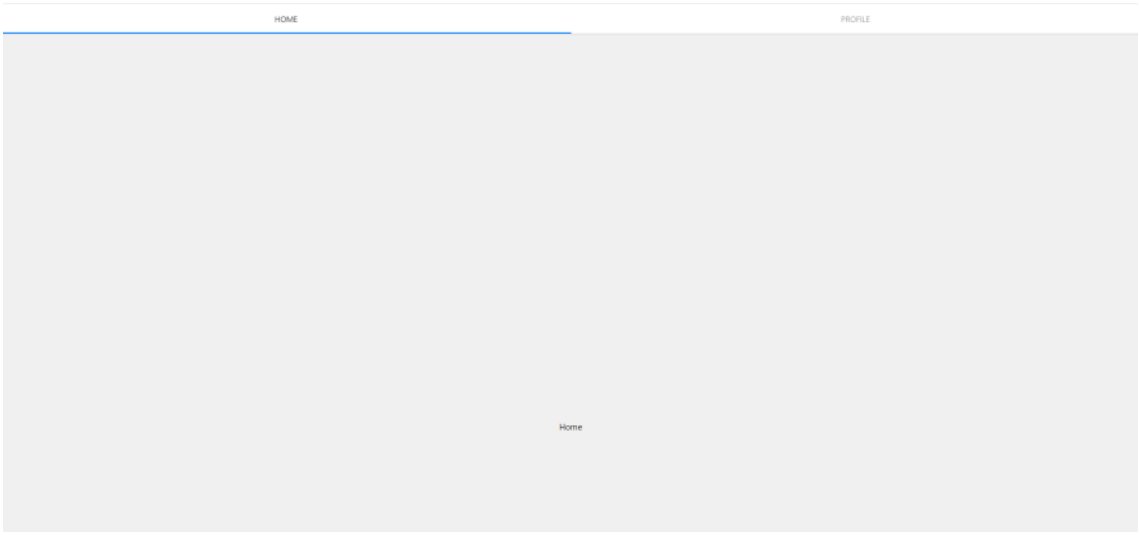


Si l'on souhaite le même type de navigation, mais avec les onglets en haut, il faut importer `@react-navigation/material-top-tabs` avec la commande `npm install @react-navigation/material-top-tabs`.

Tous droits réservés à STUDI - Reproduction interdite

À la suite de quoi, en changeant quelques lignes de l'exemple précédent, on obtient :

```
1 import { createMaterialTopTabNavigator } from '@react-navigation/material-top-tabs';
2 [...]
3 const Tab = createMaterialTopTabNavigator();
4 [...]
```



Pour ce qui concerne la déclaration du navigateur, il s'agit d'une structure très semblable à ce que l'on a vu avec `StackNavigator`, à savoir un composant parent qui prend des enfants `Screen` avec un nom et un composant.

Par défaut, les écrans sont paresseux (`lazy`), ce qui signifie que leurs composants ne seront montés qu'à la première navigation sur ces derniers. On peut passer une props `lazy={false}` à un `Tab.Navigator` si l'on souhaite pré-charger tous les écrans.

À retenir Syntaxe

- Le navigateur par onglets `TabNavigator` permet de mettre au point une autre expérience utilisateur très pratique pour une utilisation tactile. Il se décline par défaut en deux versions : une *bottom* qui permet d'afficher la barre de navigation sous les écrans comme vu dans nos exemples, ou une version *top* pour les applications qui préfèrent le menu au-dessus des écrans.
- C'est un outil que l'on peut combiner avec d'autres navigateurs, comme le navigateur en pile pour réaliser des interfaces complexes et pratiques.

Complément

- https://reactnavigation.org/docs/tab-based-navigation

Tous droits réservés à STUDI - Reproduction interdite

III Le navigateur en panneau (DrawerNavigator)

Précision sur useRoute()

useRoute est un hook qui donne accès à l'objet "route". Il va nous retourner les propriétés de l'écran et nous pourrons alors accéder depuis cet objet aux données de l'écran et nous en servir pour naviguer.

Objectifs

- Comprendre le fonctionnement d'un navigateur panneau
- Découvrir un exemple d'un navigateur panneau

Mise en situation

Le navigateur panneau (`DrawerNavigator`) est un navigateur très populaire pour la navigation mobile. Du fait d'interfaces réduites à cause de la taille de l'écran d'un téléphone et généralement en mode portrait, les designers ont dû trouver des adaptations pour offrir une navigation efficace.

De là est né le navigateur panneau, qui consiste en un panneau sortant d'un côté de l'écran pour venir se superposer à l'interface. Il est généralement déclenché par un glissement de doigt du bord vers le centre de l'écran ou via son fameux bouton burger[1], très utilisé en UX design.

Qu'est-ce qu'un navigateur panneau et quand l'utiliser ?

Très utilisé dans le webdesign, on le retrouve forcément sur l'environnement des applications mobiles. Voici un exemple de panneau déclenché par un burger menu :

(cf. nav-adv-3.mp4)

Grâce à `react-navigation`, nous allons pouvoir développer un élément semblable. Mais, avant, il faut installer une librairie additionnelle grâce à la commande `npm install @react-navigation/drawer`.

⚠ Attention

De par sa nature, il est préférable de n'avoir qu'un seul `DrawerNavigator` dans son application. En effet, en avoir plusieurs pourrait déclencher des conflits qui casseraient le fonctionnement de l'application.

Voici un exemple de code pour l'utilisation d'un navigateur panneau.

Exemple

```
1 import * as React from 'react';
2 import { Button, View } from 'react-native';
3 import { createDrawerNavigator } from '@react-navigation/drawer';
4 import { NavigationContainer } from '@react-navigation/native';
5
6 function HomeScreen({ navigation }) {
7   return (
8     <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
9       <Button
```

1. https://fr.wikipedia.org/wiki/Menu_lat%C3%A9ral

Tous droits réservés à STUDI - Reproduction interdite

```
        onPress={() => navigation.navigate('Notifications')}
        title="Go to notifications"
      />
    </View>
  );
}

function NotificationsScreen({ navigation }) {
  return (
    <View style={{ flex: 1, alignItems: 'center', justifyContent: 'center' }}>
      <Button onPress={() => navigation.goBack()} title="Go back home" />
    </View>
  );
}

const Drawer = createDrawerNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator initialRouteName="Home">
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Notifications" component={NotificationsScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

(cf. nav-adv-8.mp4)

Par défaut, le navigateur panneau se trouve sur la gauche. On peut cependant personnaliser cela en paramétrant la props `drawerPosition` à `right` ou `left`.

```
export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator
        initialRouteName="Home"
        screenOptions={{ drawerPosition: 'right' }}
      >
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Notifications" component={NotificationsScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

Il permet également par défaut le geste de *swipe* du bord vers le centre, ce qui est très pratique, mais on peut également déclencher son ouverture manuellement via un bouton dans l'interface. Nous verrons cela un peu plus tard dans un exercice.

Dans l'exemple plus haut, nous déclarons un `DrawerNavigator` avec deux écrans, `Home` et `Notifications`. On peut également naviguer vers ces écrans sans utiliser le *drawer* grâce aux boutons et à l'API traditionnelle `navigate`.

À retenir

Syntaxe

- Le navigateur en panneau est très pratique, car il n'occupe de l'espace à l'écran que lorsqu'il est affiché. C'est un incontournable de l'expérience utilisateur sur téléphone portable et il se couple parfaitement aux deux autres navigateurs fournis par `react-navigation`.

Tous droits réservés à STUDI - Reproduction interdite

- Il n'en faut qu'un seul dans toute une application, sous peine de créer des conflits de navigation, mais il s'assemble parfaitement avec n'importe quel autre type de navigateur, car, comme les autres, il prend des composants en arguments.

Complément

- https://reactnavigation.org/docs/drawer-based-navigation

Tous droits réservés à STUDI - Reproduction interdite

IV Navigation avancée

Objectifs

- Découvrir l'utilisation avancée du `StackNavigator`
- Comprendre l'affichage conditionnel
- Réaliser une fenêtre modale

Mise en situation

Maintenant que nous avons vu le fonctionnement de trois navigateurs très complémentaires, nous allons étudier les manières de gérer les interactions entre eux, notamment à travers l'exemple d'un système modal global à l'application. Nous verrons également la construction conditionnelle de la navigation, ce qui permet de gérer des cas très fréquents de navigation et qui seraient complexes sans cela (par exemple, le flux d'authentification d'un utilisateur).

Une navigation complète à base de modale globale

Imaginons le cas d'utilisation où nous souhaitons afficher une barre de navigation en bas de l'écran avec deux éléments de menu, puis, à tout moment, on souhaite pouvoir afficher une modale qui se situera par-dessus ce menu. Pour cela, il est nécessaire d'imbriquer les navigateurs de manière à ce que le `TabNavigator` soit inclus dans un `StackNavigator`.

Petit point important sur <React.Fragment> : En React Native, il est courant pour un composant de renvoyer plusieurs éléments. Les fragments nous permettent de grouper une liste d'enfants sans ajouter de nœud supplémentaire à notre écran, ce qui est plus rapide et utilise moins de mémoire. Ils facilitent également les relations parent-enfant de certains mécanismes comme Flexbox et CSS Grid. L'ajout de <View> pourrait rendre difficile le maintien de la mise en page souhaitée lors de l'extraction des composants logiques.

Voyons un exemple de cela :

Exemple

```
import * as React from "react";
import { Button, Text, View } from "react-native";
import Constants from "expo-constants";
import { NavigationContainer } from "@react-navigation/native";
import { createStackNavigator } from "@react-navigation/stack";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";

// Nos navigateurs
const MainBottomNavigation = createBottomTabNavigator();
const AppNavigation = createStackNavigator();

// Un composant simple qui affiche le nom de la route courante.
function NamedScreenComponent(props) {
  return (
    <React.Fragment>
      <Text>{props.route.name}</Text>
      <Button
        title="Ouvrir modale 1"
        onPress={() => props.navigation.navigate("Modale1")}
      />
      <Button
        title="Ouvrir modale 2"
        onPress={() => props.navigation.navigate("Modale2")}
      />
```

```
    </React.Fragment>
  );
}

// Notre navigation de bas d'écran
const BottomNav = () => (
  <MainBottomNavigation.Navigator>
    <MainBottomNavigation.Screen
      name="Premier"
      component={NamedScreenComponent}
    />
    <MainBottomNavigation.Screen
      name="Deuxieme"
      component={NamedScreenComponent}
    />
  </MainBottomNavigation.Navigator>
);

// Notre application avec notre navigateur en pile pour les modales
export default function App() {
  return (
    <View style={{ flex: 1, paddingTop: Constants.statusBarHeight }}>
      <NavigationContainer>
        <AppNavigation.Navigator>
  <AppNavigation.Screen
    name="bottom"
    options={{ headerShown: false }}
    component={BottomNav}
  />
  <AppNavigation.Screen
    name="Modale1"
    component={NamedScreenComponent}
    options={{ presentation: 'modal' }}
  />
  <AppNavigation.Screen
    name="Modale2"
    component={NamedScreenComponent}
    options={{ presentation: 'modal' }}
  />
</AppNavigation.Navigator>
      </NavigationContainer>
    </View>
  );
}

```

(cf. nav-adv-9.mp4)

L'exemple ci-dessus utilise comme navigateur principal un `StackNavigator`, nommé `AppNavigation`, avec comme premier écran un composant de navigation par onglets `TabNavigator`, appelé `MainBottomNavigation`, définissant deux éléments. De cette manière, la navigation du bas (`MainBottomNavigation`) et ses éléments feront partie de la boîte où s'affichent les écrans de notre navigateur en pile (`AppNavigation`).

Une fois que nous avons cela, si nous souhaitons ajouter une modale, il suffit de la déclarer à la suite de notre écran `MainBottomNavigation` dans la déclaration de notre navigateur en pile (`AppNavigation`).

Tous droits réservés à STUDI - Reproduction interdite

On peut par la suite ajouter autant de modales qu'on le souhaite et adapter cette composition comme bon nous semble. Petit plus : nous avons réglé l'animation écran par écran grâce à l'option `presentation: 'modal'`. Sur iOS, les modales apparaissent depuis le bas de l'écran et restent *swipeables* vers le bas, un pattern très commun. Cela se verra essentiellement sur iOS, où les modales apparaîtront depuis le bas de l'écran et seront *swipeables* vers le bas, un pattern très commun sur iOS.

Utiliser un affichage conditionnel pour gérer un flux d'authentification

Un autre cas d'utilisation avancé et très pratique est la notion de « navigation dynamique ». En effet, imaginons que nous ayons une application où il y a deux modes : connecté/déconnecté. C'est un cas d'usage très classique et, grâce à `react-navigation`, on peut y répondre très simplement, ce qui n'a pas toujours été le cas.

Comment est-ce que ça fonctionne ?

Le principe est simple : en reprenant l'exemple vu précédemment, nous allons scinder notre navigation en deux parties distinctes, que nous afficherons conditionnellement selon l'état de connexion de l'utilisateur. Nous afficherons également un écran de chargement le temps que l'on récupère cet état. Pour cela, nous allons simuler une requête asynchrone à un état grâce à `setTimeout` afin de voir le *loader* s'afficher, puis nous afficherons aléatoirement le mode connecté ou déconnecté.

Exemple

```
import * as React from "react";
import { ActivityIndicator, Text, View } from "react-native";
import Constants from "expo-constants";
import { NavigationContainer } from "@react-navigation/native";
import { createStackNavigator } from "@react-navigation/stack";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs";

// Nos navigateurs
const MainBottomNavigation = createBottomTabNavigator();
const AppNavigation = createStackNavigator();

// Un composant simple qui affiche le nom de la route courante.
function NamedScreenComponent(props) {
  return (
    <React.Fragment>
      <Text>{props.route.name}</Text>
    </React.Fragment>
  );
}

// Notre navigation de bas d'écran
const BottomNav = () => (
  <MainBottomNavigation.Navigator>
    <MainBottomNavigation.Screen
      name="Premier"
      component={NamedScreenComponent}
    />
    <MainBottomNavigation.Screen
      name="Deuxieme"
      component={NamedScreenComponent}
    />
  </MainBottomNavigation.Navigator>
);

// Notre application avec notre navigateur en pile pour les modales
export default function App() {
```

```
  const [loginState, setLoginState] = React.useState("unknown");

  React.useEffect(() => {
    // On affiche aléatoirement l'état de connexion après 2 secondes
    setTimeout(() => {
      setLoginState(Math.random() > 0.5 ? "loggedIn" : "loggedOut");
    }, 2000);
  }, []);

  // On affiche le chargement au démarrage le temps de récupérer l'état de
connexion
  if (loginState === "unknown") {
    return (
      <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
        <ActivityIndicator size="large" />
        <Text>Chargement en cours...</Text>
      </View>
    );
  }

  return (
    <View style={{ flex: 1, paddingTop: Constants.statusBarHeight }}>
      <NavigationContainer>
        <AppNavigation.Navigator>
  {loginState === "loggedIn" ? (
    <AppNavigation.Screen
      name="bottom"
      options={{ headerShown: false }}
      component={BottomNav}
    />
  ) : (
    <AppNavigation.Screen
      name="Écran de connexion"
      options={{ headerShown: false }}
      component={NamedScreenComponent}
    />
  )}
  <AppNavigation.Screen
    name="Modale1"
    component={NamedScreenComponent}
    options={{ presentation: 'modal' }}
  />
  <AppNavigation.Screen
    name="Modale2"
    component={NamedScreenComponent}
    options={{ presentation: 'modal' }}
  />
</AppNavigation.Navigator>
      </NavigationContainer>
    </View>
  );
}

```

(cf. nav-adv-10.mp4)

En rafraîchissant l'application, on pourra constater, qu'aléatoirement, une fois sur deux, il y aura le panneau de connexion ou le panneau connecté. Bien évidemment, dans une vraie application, il faut se baser sur un état de connexion vérifiée avec son back-end.

Tous droits réservés à STUDI - Reproduction interdite

À retenir Syntaxe

- Il est possible de composer une navigation très avancée en combinant les trois navigateurs de `react-navigation`, cela nous permet notamment de gérer un système de modales et de contrôler la superposition des éléments dans l'interface.
- Pour aller plus loin, on peut également composer conditionnellement la navigation en utilisant de simples structures conditionnelles. Cela nous permettra de créer des scénarios complexes, par exemple pour l'*onboarding* d'un utilisateur ou bien pour masquer des éléments de l'interface à des membres qui n'ont pas payé d'accès premium à une fonctionnalité.

Complément

- https://reactnavigation.org/docs/modal
- https://reactnavigation.org/docs/auth-flow

Tous droits réservés à STUDI - Reproduction interdite

V Synthèse

A. Flashcards

TabNavigator

Navigateur affichant un menu à onglets pour basculer entre écrans par clic ou swipe.

DrawerNavigator

Navigateur présentant un panneau latéral superposé, déclenché par swipe ou bouton burger.

Bottom tabs

Menu à onglets positionné en bas d'écran, très utilisé sur mobile.

Top tabs

Menu à onglets positionné en haut d'écran, alternative au bottom.

Chargement lazy

Montage des composants écran uniquement à la première navigation vers ceux-ci.

Bouton burger

Icône trois lignes horizontales déclenchant l'ouverture du panneau latéral.

Modale globale

Écran superposé à toute l'interface via StackNavigator encapsulant TabNavigator.

Navigation conditionnelle

Affichage dynamique de flux navigation selon état utilisateur ou contexte.

Imbrication navigateurs

Combinaison de plusieurs types de navigateurs pour interfaces complexes.

Option presentation modal

Paramètre définissant animation modale depuis bas écran sur iOS.

React Fragment

Groupement d'éléments enfants sans ajout de nœud supplémentaire à l'écran.

drawerPosition

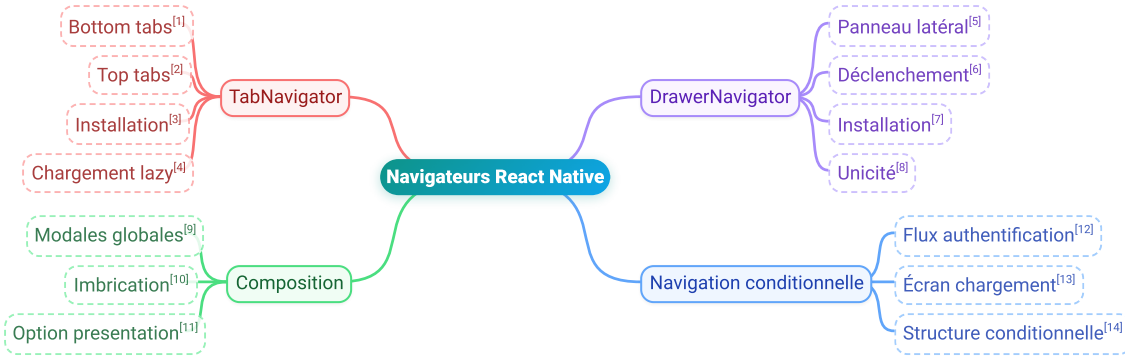
Props définissant position du panneau latéral, left ou right.

La théorie en Bref

Fondamental

Les **navigateurs** dans React Native structurent l'expérience utilisateur mobile. Le **TabNavigator** affiche un menu à onglets (en haut ou en bas) permettant de basculer entre écrans par clic ou swipe, idéal pour les interfaces tactiles. Le **DrawerNavigator** propose un panneau latéral qui se superpose à l'écran, déclenché par swipe ou bouton burger, pratique pour économiser l'espace. Le **StackNavigator** gère la navigation en pile et peut encapsuler les autres navigateurs pour créer des **modales globales**. La **navigation conditionnelle** permet d'afficher dynamiquement des flux différents selon l'état utilisateur, par exemple pour gérer l'authentification. La **composition** de ces navigateurs offre des parcours utilisateurs complets et professionnels.

Tous droits réservés à STUDI - Reproduction interdite



1. Menu en bas d'écran, très utilisé (ex: Instagram).
2. Menu en haut d'écran, alternative au bottom.
3. Nécessite @react-navigation/bottom-tabs ou material-top-tabs.
4. Écrans montés à la première navigation par défaut.
5. Se superpose à l'interface, gauche ou droite.
6. Swipe du bord ou bouton burger.
7. Nécessite @react-navigation/drawer.
8. Un seul DrawerNavigator par application pour éviter conflits.
9. StackNavigator encapsule TabNavigator pour superposer modales.
10. Combiner navigateurs pour interfaces complexes.
11. Définit l'animation modale sur iOS.
12. Affichage dynamique selon état connecté/déconnecté.
13. Loader pendant récupération état utilisateur.
14. Utilise if/else ou ternaire pour composer navigation.

Navigateurs React Native

Les conseils pro

Conseil

Comment choisir le type de navigateur adapté à son interface mobile ?

- Identifier les actions principales utilisateur et leur fréquence d'usage.
- Utiliser TabNavigator pour 3 à 5 fonctions essentielles accessibles en permanence.
- Réserver DrawerNavigator aux menus secondaires ou actions moins fréquentes.
- Combiner StackNavigator pour gérer modales et flux séquentiels au-dessus.

Comment structurer une navigation complexe avec plusieurs navigateurs imbriqués ?

- Définir le navigateur principal selon le flux dominant de l'application.
- Encapsuler TabNavigator ou DrawerNavigator dans StackNavigator pour modales globales.
- Documenter l'arborescence de navigation pour faciliter maintenance et évolutions.
- Tester les transitions entre écrans sur iOS et Android pour valider cohérence.

Comment implémenter un flux d'authentification avec navigation conditionnelle ?

Tous droits réservés à STUDI - Reproduction interdite

- Créer un état global stockant le statut connecté/déconnecté de l'utilisateur.
- Afficher un écran de chargement pendant vérification asynchrone du statut.
- Composer conditionnellement les écrans selon état avec structure if/else ou ternaire.
- Éviter de stocker données sensibles côté client, valider toujours côté back-end.

Comment optimiser les performances d'un TabNavigator avec nombreux écrans ?

- Activer le chargement lazy par défaut pour monter écrans à la demande.
- Désactiver lazy uniquement pour écrans nécessitant pré-chargement de données critiques.
- Limiter le nombre d'onglets affichés simultanément à 5 maximum pour lisibilité.
- Mesurer temps de montage composants et optimiser rendus lourds si nécessaire.

Comment garantir une expérience utilisateur cohérente avec DrawerNavigator ?

- Limiter à un seul DrawerNavigator par application pour éviter conflits de navigation.
- Positionner le panneau selon conventions plateforme, gauche par défaut.
- Activer le swipe du bord pour ouverture intuitive sans bouton visible.
- Tester l'accessibilité du panneau sur petits écrans et en mode paysage.

Tous droits réservés à STUDI - Reproduction interdite

VI Auto évaluation

A. Exercice : Concevoir la navigation d'une application mobile de gestion d'événements

Mise en situation

Vous venez d'intégrer l'équipe de développement mobile de **EventFlow**, une start-up qui développe une application React Native de gestion d'événements culturels. L'application permet aux utilisateurs de découvrir des événements, de gérer leurs réservations et de consulter leur profil.

La lead développeuse, Nadia, vous confie la mission de restructurer la navigation de l'application. Actuellement, l'application utilise uniquement un `StackNavigator` basique, ce qui rend l'expérience utilisateur peu intuitive : les utilisateurs doivent revenir en arrière à chaque fois pour accéder à une autre section.

Nadia vous transmet les spécifications suivantes :

Spécifications fonctionnelles

L'application doit comporter **4 sections principales** accessibles en permanence :

- **Découvrir** : liste des événements à venir
- **Mes réservations** : historique et réservations actives
- **Recherche** : recherche avancée par critères
- **Profil** : informations et paramètres utilisateur

En complément, l'application doit proposer :

- Un **panneau latéral** contenant les liens vers les mentions légales, les CGU, le centre d'aide et le bouton de déconnexion
- Une **modale de confirmation** qui s'affiche lorsqu'un utilisateur valide une réservation, superposée à l'ensemble de l'interface
- Un **flux d'authentification** : les utilisateurs non connectés ne doivent voir qu'un écran de connexion, sans accès aux onglets ni au panneau latéral

Contraintes techniques

- L'application utilise **React Native** avec **react-navigation v6**
- Les modules disponibles sont : `@react-navigation/native`, `@react-navigation/bottom-tabs`, `@react-navigation/stack`, `@react-navigation/drawer`
- Le chargement des écrans doit être **lazy par défaut** pour optimiser les performances
- Le panneau latéral doit être positionné à **gauche** et accessible par **swipe** ou via un **bouton burger**
- Un seul `DrawerNavigator` est autorisé dans toute l'application
- La modale de confirmation doit utiliser l'option `presentation: 'modal'` pour bénéficier de l'animation native iOS

Question 1

[solution n°1 p. 24]

Identifiez et justifiez le type de navigateur à utiliser pour chacune des fonctionnalités demandées par Nadia (sections principales, panneau latéral, modale de confirmation). Expliquez comment vous les imbriquez les uns dans les autres pour obtenir une architecture de navigation cohérente.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

[solution n°2 p. 24]

Rédigez le squelette de code (structure JSX uniquement, sans logique métier) qui met en place l'architecture de navigation complète : le StackNavigator racine contenant le DrawerNavigator, le TabNavigator avec les 4 onglets, et l'écran modal de confirmation de réservation. Justifiez l'ordre de déclaration des écrans.

Question 3

[solution n°3 p. 25]

Nadia vous demande maintenant d'intégrer le flux d'authentification : les utilisateurs non connectés ne doivent voir qu'un écran de connexion, sans accès aux onglets ni au panneau latéral. Décrivez la stratégie de navigation conditionnelle à mettre en place et identifiez les pièges potentiels à éviter.

Question 4

[solution n°4 p. 26]

L'application EventFlow connaît des ralentissements au démarrage : les 4 écrans du TabNavigator se chargent simultanément. Proposez une stratégie d'optimisation des performances en exploitant les mécanismes de chargement de react-navigation, et expliquez dans quel cas précis vous désactiveriez le chargement lazy.

B. Quiz

Exercice 1 : TabNavigator et DrawerNavigator

[solution n°5 p. 26]

Exercice

Par défaut, tous les écrans d'un TabNavigator sont montés au démarrage de l'application, même si l'utilisateur ne les a pas encore visités.

- Vrai
- Faux

Exercice

Vous démarrez un nouveau projet React Native et souhaitez intégrer un menu à onglets en bas d'écran. Quelle commande d'installation est correcte ?

- npm install @react-navigation/bottom-tabs
- npm install @react-navigation/native @react-navigation/bottom-tabs
- npm install @react-navigation/tabs
- npm install @react-navigation/native @react-navigation/drawer

Exercice

Dans votre application, vous souhaitez que le panneau latéral du DrawerNavigator s'ouvre depuis la droite de l'écran. Quelle configuration devez-vous appliquer ?

- Passer la props `drawerSide='right'` au composant `Drawer.Navigator`
- Passer `screenOptions={{ drawerPosition: 'right' }}` au composant `Drawer.Navigator`

Tous droits réservés à STUDI - Reproduction interdite

Passer la props `position='right'` directement sur chaque `Drawer.Screen`

Modifier la props `initialRouteName` pour cibler l'écran de droite

Exercice

Parmi les affirmations suivantes concernant le DrawerNavigator, lesquelles sont correctes ?

A Il est recommandé de n'utiliser qu'un seul DrawerNavigator par application

B Son panneau peut être déclenché par un swipe depuis le bord de l'écran

C Il peut être positionné à gauche ou à droite de l'écran

D Il est impossible de naviguer entre ses écrans autrement que via le panneau latéral

E Son installation nécessite le module @react-navigation/drawer

Exercice

Associez chaque navigateur à sa caractéristique principale telle que décrite dans la leçon.

A Material Top Tabs **B** DrawerNavigator **C** TabNavigator **D** StackNavigator

Affiche un menu en ligne permettant de basculer entre écrans par clic ou swipe	Présente un panneau latéral qui se superpose à l'interface	Gère la navigation en pile et peut encapsuler les autres navigateurs pour les modales	Variante du navigateur à onglets positionnée en haut de l'écran

Exercice 7 : Composition et navigation avancée

[solution n°6 p. 28]

Exercice

Pour afficher une modale par-dessus un menu à onglets, il suffit de déclarer la modale directement dans le TabNavigator.

Vrai

Faux

Tous droits réservés à STUDI - Reproduction interdite

Exercice

Dans l'exemple de navigation conditionnelle de la leçon, que se passe-t-il pendant les 2 secondes simulant la récupération de l'état de connexion ?

- L'application affiche directement l'écran de connexion par défaut
- Un composant ActivityIndicator et un texte de chargement sont affichés
- Le NavigationContainer est rendu sans aucun écran visible
- L'application affiche aléatoirement l'un des deux flux de navigation

Exercice

Parmi les affirmations suivantes sur la composition de navigateurs dans React Native, lesquelles sont correctes ?

- [ ] **A** Un TabNavigator peut être utilisé comme composant d'un écran dans un StackNavigator
- [ ] **B** La navigation conditionnelle peut s'appuyer sur des structures if/else ou ternaires
- [ ] **C** Il est impossible de naviguer vers un écran du StackNavigator depuis un composant imbriqué dans le TabNavigator
- [ ] **D** L'option `presentation: 'modal'` définit une animation depuis le bas de l'écran, principalement visible sur iOS
- [ ] **E** React.Fragment permet de grouper des éléments enfants sans ajouter de nœud supplémentaire à l'écran

Exercice

Remettez dans le bon ordre les étapes permettant de mettre en place une modale globale au-dessus d'un menu à onglets dans React Native.

Tous droits réservés à STUDI - Reproduction interdite

A Ajouter les écrans modaux à la suite dans le StackNavigator avec l'option `presentation: 'modal'`

B Instancier un StackNavigator (`createStackNavigator`) et un TabNavigator (`createBottomTabNavigator`)

C Déclarer le TabNavigator comme composant du premier écran du StackNavigator

D Envelopper le tout dans un `NavigationContainer` et exporter le composant App

E Créer les composants d'écran nécessaires (écrans du tab et écran modale)

Réponse : ___ ___ ___ ___ ___

Exercice

Associez chaque concept de navigation avancée à sa description fonctionnelle.

A Modale globale | **B** Chargement lazy | **C** Navigation conditionnelle | **D** React.Fragment

Affichage dynamique de flux différents selon l'état de l'utilisateur, par exemple connecté ou déconnecté	Écran superposé à toute l'interface, rendu possible par l'encapsulation du TabNavigator dans un StackNavigator	Groupement d'éléments enfants dans un composant sans ajouter de nœud supplémentaire à l'arbre de rendu	Montage des composants écran uniquement lors de la première navigation vers ceux-ci

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

[exercice p. 19] Solution n°1

Réponse attendue

Pour les **4 sections principales** (Découvrir, Mes réservations, Recherche, Profil), vous utilisez un **TabNavigator** (`createBottomTabNavigator`) car ces sections doivent être accessibles en permanence via un menu en bas d'écran, ce qui correspond au cas d'usage classique d'un navigateur par onglets sur mobile. Pour le **panneau latéral** (mentions légales, CGU, aide, déconnexion), vous utilisez un **DrawerNavigator** (`createDrawerNavigator`) positionné à gauche, car il s'agit de fonctionnalités secondaires qui ne nécessitent pas d'être visibles en permanence. Pour la **modale de confirmation**, vous utilisez un **StackNavigator** (`createStackNavigator`) avec l'option `presentation: 'modal'`, car la modale doit se superposer à l'ensemble de l'interface. L'imbrication suit cette logique : le `StackNavigator` est le navigateur racine, il contient comme premier écran le `DrawerNavigator`, qui lui-même encapsule le `TabNavigator` avec les 4 onglets. Les écrans modaux sont déclarés au niveau du `StackNavigator` racine pour pouvoir se superposer à toute l'interface.

Éléments de solution

- **TabNavigator** : adapté aux **3 à 5 fonctions essentielles** accessibles en permanence, usage tactile optimal
- **DrawerNavigator** : réservé aux **menus secondaires**, économise l'espace écran, un seul par application pour éviter les conflits
- **StackNavigator** : permet la **superposition d'écrans** (modales) au-dessus des autres navigateurs
- **Principe d'imbrication** : le navigateur le plus englobant (StackNavigator) encapsule les autres, ce qui permet aux modales de se superposer à l'ensemble de l'interface
- L'option `presentation: 'modal'` sur les écrans du StackNavigator déclenche l'animation native depuis le bas sur iOS
- L'ordre d'imbrication est crucial : inverser DrawerNavigator et StackNavigator empêcherait la modale de se superposer au panneau latéral

[exercice p. 20] Solution n°2

Réponse attendue

Le squelette de code doit déclarer trois navigateurs : `const Stack = createStackNavigator()`, `const Drawer = createDrawerNavigator()` et `const Tab = createBottomTabNavigator()`. Le composant `TabNav` regroupe les 4 onglets (Découvrir, Réservations, Recherche, Profil) dans un `Tab.Navigator`. Le composant `DrawerNav` encapsule `TabNav` comme écran principal et ajoute les écrans secondaires (MentionsLégales, CGU, Aide). Enfin, le composant `App` déclare le `Stack.Navigator` racine avec `DrawerNav` comme premier écran (option `headerShown: false` pour éviter un double header) et l'écran `ConfirmationReservation` avec `presentation: 'modal'`. L'ordre de déclaration est important : le premier écran du `Stack.Navigator` est celui affiché par défaut, et les écrans modaux sont déclarés après pour pouvoir se superposer à l'ensemble de la navigation.

Tous droits réservés à STUDI - Reproduction interdite

Éléments de solution

- **Trois niveaux d'imbrication** : `Stack.Navigator` > `Drawer.Navigator` > `Tab.Navigator`
- Le `Tab.Navigator` déclare les **4 écrans principaux** avec leurs composants respectifs
- Le `Drawer.Navigator` prend `TabNav` comme composant de son premier écran, ce qui permet au panneau latéral de coexister avec les onglets
- Le `Stack.Navigator` racine déclare `DrawerNav` en premier écran et les modales ensuite
- L'option `headerShown: false` sur l'écran `DrawerNav` dans le Stack évite la superposition de barres de titre
- L'option `presentation: 'modal'` sur l'écran de confirmation active l'animation modale native
- Le chargement **lazy** est activé par défaut sur le `Tab.Navigator`, inutile de le configurer explicitement
- La prop `drawerPosition: 'left'` est la valeur par défaut, mais peut être précisée explicitement pour la lisibilité du code

[exercice p. 20] Solution n°3

Réponse attendue

La stratégie repose sur un **affichage conditionnel** des écrans au sein du `Stack.Navigator` racine. Vous créez un état `loginState` avec trois valeurs possibles : `'unknown'` (chargement), `'loggedIn'` (connecté) et `'loggedOut'` (déconnecté). Pendant la phase `'unknown'`, vous affichez un écran de chargement avec un `ActivityIndicator` pour indiquer à l'utilisateur que la vérification est en cours. Ensuite, dans le `Stack.Navigator`, vous utilisez une structure conditionnelle ternaire : si `loginState === 'loggedIn'`, vous affichez l'écran contenant le `DrawerNavigator` (et donc les onglets) ; sinon, vous affichez uniquement l'écran de connexion. Les écrans modaux restent accessibles uniquement en mode connecté. Les pièges à éviter sont : ne pas gérer l'état `'unknown'` (ce qui provoquerait un flash d'écran), stocker des données sensibles côté client au lieu de valider côté back-end, et oublier que la navigation conditionnelle repose sur le re-rendu React — il ne faut pas utiliser `navigation.navigate` pour basculer entre les flux connecté et déconnecté.

Éléments de solution

- **Trois états de connexion** : `'unknown'`, `'loggedIn'`, `'loggedOut'` gérés via `React.useState`
- **Écran de chargement** : affiché pendant la vérification asynchrone du statut, évite le flash d'interface
- **Structure conditionnelle** : ternaire dans le JSX du `Stack.Navigator` pour composer dynamiquement les écrans disponibles
- **Piège n°1** : ne pas gérer l'état intermédiaire `'unknown'` provoque un affichage incorrect au démarrage
- **Piège n°2** : utiliser `navigation.navigate('Login')` au lieu de la navigation conditionnelle — cela permettrait à l'utilisateur de revenir en arrière vers les écrans protégés
- **Piège n°3** : stocker le token d'authentification uniquement côté client sans validation serveur, ce qui pose un risque de sécurité
- **Piège n°4** : oublier de placer les écrans modaux dans la branche `'loggedIn'` uniquement, ce qui les rendrait accessibles sans authentification
- La navigation conditionnelle fonctionne grâce au **re-rendu React** : quand l'état change, les écrans disponibles sont recalculés automatiquement par `react-navigation`

Tous droits réservés à STUDI - Reproduction interdite

[exercice p. 20]Solution n°4

Réponse attendue

Par défaut, le `TabNavigator` de react-navigation utilise le **chargement lazy** : les composants des écrans ne sont montés qu'à la première navigation vers ceux-ci. Si les 4 écrans se chargent simultanément, cela signifie probablement que la prop `lazy` a été explicitement désactivée (`lazy={false}`) sur le `Tab.Navigator`, ou que des effets de bord dans les composants parents déclenchent le montage prématuré de tous les écrans. La première action est de vérifier que `lazy` n'est pas désactivé et de le remettre à `true` (ou de supprimer la prop, car c'est la valeur par défaut). Vous ne désactiveriez le chargement lazy que dans un cas précis : lorsqu'un écran nécessite un **pré-chargement de données critiques** qui doivent être disponibles instantanément dès que l'utilisateur navigue vers cet onglet, par exemple un écran de messagerie en temps réel où un délai de chargement serait inacceptable. Dans tous les autres cas, le lazy loading doit rester activé pour réduire le temps de montage initial.

Éléments de solution

- Le **chargement lazy** est activé **par défaut** sur le `Tab.Navigator` : les écrans ne sont montés qu'à la première navigation
- Si tous les écrans se chargent au démarrage, vérifier que `lazy={false}` n'a pas été ajouté au `Tab.Navigator`
- **Cas de désactivation du lazy** : écran nécessitant des données pré-chargées (messagerie temps réel, tableau de bord avec données critiques)
- **Limiter le nombre d'onglets** à 5 maximum pour la lisibilité et les performances
- Mesurer le temps de montage des composants avec les outils de profiling React pour identifier les rendus lourds
- Optimiser les composants lourds avec `React.memo` ou `useMemo` pour éviter les re-rendus inutiles
- Le chargement lazy ne concerne que le **montage initial** : une fois monté, un écran reste en mémoire tant que le navigateur existe

Solution n°5

[exercice p. 20]

Exercice

Par défaut, tous les écrans d'un TabNavigator sont montés au démarrage de l'application, même si l'utilisateur ne les a pas encore visités.

Vrai

✓ Faux

Par défaut, les écrans d'un TabNavigator sont en mode 'lazy' : ils ne sont montés qu'à la première navigation vers eux. Ce comportement peut être désactivé en passant la props `lazy={false}` au composant `Tab.Navigator`, mais ce n'est pas le comportement par défaut. Confondre ce comportement avec un pré-chargement systématique est une erreur fréquente.

Tous droits réservés à STUDI - Reproduction interdite

Exercice

Vous démarrez un nouveau projet React Native et souhaitez intégrer un menu à onglets en bas d'écran. Quelle commande d'installation est correcte ?

npm install @react-navigation/bottom-tabs

✓ npm install @react-navigation/native @react-navigation/bottom-tabs

npm install @react-navigation/tabs

npm install @react-navigation/native @react-navigation/drawer

Pour utiliser le navigateur par onglets en bas d'écran, il faut s'assurer d'avoir `@react-navigation/native` (le cœur de la librairie) ET `@react-navigation/bottom-tabs` (le module spécifique). La commande complète est donc `npm install @react-navigation/native @react-navigation/bottom-tabs`. Installer uniquement `@react-navigation/bottom-tabs` sans `@react-navigation/native` ne suffit pas si ce dernier n'est pas déjà présent dans le projet.

Exercice

Dans votre application, vous souhaitez que le panneau latéral du DrawerNavigator s'ouvre depuis la droite de l'écran. Quelle configuration devez-vous appliquer ?

Passer la props `drawerSide='right'` au composant `Drawer.Navigator`

✓ Passer `screenOptions={{ drawerPosition: 'right' }}` au composant `Drawer.Navigator`

Passer la props `position='right'` directement sur chaque `Drawer.Screen`

Modifier la props `initialRouteName` pour cibler l'écran de droite

La position du panneau latéral se configure via la props `screenOptions` du composant `Drawer.Navigator`, en y passant l'objet `{ drawerPosition: 'right' }`. La props s'appelle bien `drawerPosition` et non `drawerSide` ou `position`. La props `initialRouteName` ne concerne que l'écran affiché au démarrage et n'a aucun effet sur la position du panneau.

Exercice

Parmi les affirmations suivantes concernant le DrawerNavigator, lesquelles sont correctes ?

A Il est recommandé de n'utiliser qu'un seul DrawerNavigator par application

B Son panneau peut être déclenché par un swipe depuis le bord de l'écran

Tous droits réservés à STUDI - Reproduction interdite

C Il peut être positionné à gauche ou à droite de l'écran

D Il est impossible de naviguer entre ses écrans autrement que via le panneau latéral

E Son installation nécessite le module @react-navigation/drawer

Trois affirmations sont correctes : il est fortement recommandé de n'avoir qu'un seul DrawerNavigator (plusieurs créeraient des conflits), le swipe depuis le bord est activé par défaut, la position est configurable via `drawerPosition`, et le module `@react-navigation/drawer` est requis. En revanche, il est tout à fait possible de naviguer entre les écrans du drawer via l'API `navigation.navigate()` et des boutons classiques, comme le montre l'exemple de la leçon avec `navigation.navigate('Notifications')`.

Exercice

Associez chaque navigateur à sa caractéristique principale telle que décrite dans la leçon.

Affiche un menu en ligne permettant de basculer entre écrans par clic ou swipe	Présente un panneau latéral qui se superpose à l'interface	Gère la navigation en pile et peut encapsuler les autres navigateurs pour les modales	Variante du navigateur à onglets positionnée en haut de l'écran
TabNavigator	DrawerNavigator	StackNavigator	Material Top Tabs

Le TabNavigator (bottom tabs) affiche un menu horizontal en bas d'écran pour naviguer par clic ou swipe. Le DrawerNavigator propose un panneau latéral superposé, déclenché par swipe ou bouton burger. Le StackNavigator gère la navigation en pile et sert de conteneur principal pour encapsuler les autres navigateurs et gérer les modales. Le Material Top Tabs est la variante du TabNavigator positionnée en haut, installée via `@react-navigation/material-top-tabs`.

Solution n°6

[exercice p. 21]

Exercice

Pour afficher une modale par-dessus un menu à onglets, il suffit de déclarer la modale directement dans le TabNavigator.

Vrai

✓ Faux

Tous droits réservés à STUDI - Reproduction interdite

Pour qu'une modale se superpose à l'ensemble de l'interface, y compris au menu à onglets, il est nécessaire d'encapsuler le TabNavigator dans un StackNavigator. La modale est ensuite déclarée comme un écran supplémentaire dans ce StackNavigator, après l'écran contenant le TabNavigator. Déclarer la modale directement dans le TabNavigator ne permettrait pas cette superposition globale.

Exercice

Dans l'exemple de navigation conditionnelle de la leçon, que se passe-t-il pendant les 2 secondes simulant la récupération de l'état de connexion ?

L'application affiche directement l'écran de connexion par défaut

✓ Un composant ActivityIndicator et un texte de chargement sont affichés

Le NavigationContainer est rendu sans aucun écran visible

L'application affiche aléatoirement l'un des deux flux de navigation

Pendant la phase de récupération de l'état de connexion (simulée par un `setTimeout` de 2 secondes), l'état `loginState` vaut `'unknown'`. Dans ce cas, le composant retourne une vue avec un `ActivityIndicator` (spinner) et le texte 'Chargement en cours...', sans afficher le `NavigationContainer`. Ce n'est qu'après les 2 secondes que l'état est mis à jour et que la navigation appropriée est rendue.

Exercice

Parmi les affirmations suivantes sur la composition de navigateurs dans React Native, lesquelles sont correctes ?

A Un TabNavigator peut être utilisé comme composant d'un écran dans un StackNavigator

B La navigation conditionnelle peut s'appuyer sur des structures if/else ou ternaires

C Il est impossible de naviguer vers un écran du StackNavigator depuis un composant imbriqué dans le TabNavigator

D ☐ L'option `presentation: 'modal'` définit une animation depuis le bas de l'écran, principalement visible sur iOS

E ☐ React.Fragment permet de grouper des éléments enfants sans ajouter de nœud supplémentaire à l'écran

Tous droits réservés à STUDI - Reproduction interdite

Trois affirmations sont correctes : un TabNavigator peut bien être passé comme `component` d'un écran dans un StackNavigator (c'est le cœur du pattern modale globale) ; la navigation conditionnelle utilise des structures if/else ou ternaires ; l'option `presentation: 'modal'` produit une animation depuis le bas, surtout visible sur iOS ; et React.Fragment groupe des éléments sans nœud DOM supplémentaire. En revanche, il est tout à fait possible de naviguer vers un écran du StackNavigator depuis un composant imbriqué dans le TabNavigator, comme le montre l'exemple avec `navigation.navigate('Modale1')`.

Exercice

Remettez dans le bon ordre les étapes permettant de mettre en place une modale globale au-dessus d'un menu à onglets dans React Native.

A Créer les composants d'écran nécessaires (écrans du tab et écran modale)

B Instancier un StackNavigator (`createStackNavigator`) et un TabNavigator (`createBottomTabNavigator`)

C Déclarer le TabNavigator comme composant du premier écran du StackNavigator

D Ajouter les écrans modaux à la suite dans le StackNavigator avec l'option `presentation: 'modal'`

E Envelopper le tout dans un `NavigationContainer` et exporter le composant App

La logique de construction suit cet ordre : on commence par créer les composants d'écran, puis on instancie les deux navigateurs. Ensuite, on imbrique le TabNavigator comme composant du premier écran du StackNavigator, ce qui lui permet de s'afficher dans la zone d'écran du navigateur en pile. On ajoute alors les modales comme écrans supplémentaires du StackNavigator avec l'option `presentation: 'modal'`. Enfin, on enveloppe l'ensemble dans le `NavigationContainer` pour que la navigation soit fonctionnelle.

Exercice

Associez chaque concept de navigation avancée à sa description fonctionnelle.

Affichage dynamique de flux différents selon l'état de l'utilisateur, par exemple connecté ou déconnecté	**Écran superposé à toute l'interface, rendu possible par l'encapsulation du TabNavigator dans un StackNavigator**	**Groupement d'éléments enfants dans un composant sans ajouter de nœud supplémentaire à l'arbre de rendu**	**Montage des composants écran uniquement lors de la première navigation vers ceux-ci**
Navigation conditionnelle	Modale globale	React.Fragment	Chargement lazy

Tous droits réservés à STUDI - Reproduction interdite

La navigation conditionnelle repose sur des structures if/else ou ternaires pour composer dynamiquement les écrans selon un état (connexion, onboarding, accès premium). La modale globale nécessite d'encapsuler le TabNavigator dans un StackNavigator, les modales étant déclarées comme écrans supplémentaires de ce dernier. React.Fragment permet de retourner plusieurs éléments depuis un composant sans créer de nœud View supplémentaire, ce qui préserve la mise en page Flexbox. Le chargement lazy est le comportement par défaut du TabNavigator : les écrans ne sont montés qu'à la première visite.

Tous droits réservés à STUDI - Reproduction interdite